Chapter 9 — CSS Grid

Flexbox (Chapter 8) arranges items along **one** axis, sized by their content. **CSS Grid** defines **rows and columns at the same time**, then places items into the cells — or lets the browser place them. It is the tool for page structure, card galleries, dashboards, forms with aligned labels, and anything a designer drew on a grid. Grid has worked in every major browser since 2017 (Baseline widely available); `subgrid` is Baseline since September 2023.

In this chapter: 9.1 Grid vocabulary · 9.2 Defining tracks and the `fr` maths · 9.3 `repeat()`, `auto-fill` vs `auto-fit` (responsive grids without media queries) · 9.4 Placing items (lines, `span`, named lines, `grid-template-areas`, overlap) · 9.5 The implicit grid and auto-placement · 9.6 Alignment · 9.7 Subgrid · 9.8 Grid vs Flexbox · 9.9 Patterns · 9.10 Debugging, pitfalls and interview prep.

As in Chapters 7–8, every example is a page in `code/` with a screenshot in `img/`: containers have a **dashed purple border**, items a **solid blue outline**, **red badges** mark numbered lines, **red numbers** are track sizes read back from the browser. The `.mjs` scripts read `getComputedStyle(grid).gridTemplateColumns` — the **resolved** track sizes in pixels, the browser's own answer — and compare them with the hand calculation.

9.1 Grid vocabulary

What it is

`display: grid` (or `inline-grid`) makes a **grid container**; its direct children become **grid items**. The container defines an invisible grid (like a spreadsheet whose columns you size with maths):

Term	Meaning
Grid line	A dividing line between tracks, numbered from **1** at the start edge and backwards from **−1** (the last line) at the end edge. 3 columns → 4 column lines (`1fr 1fr 1fr` → 5).
Track	The space between two adjacent lines: a **column** or a **row**.
Cell	One row × one column — the smallest unit.
Grid area	Any rectangle of cells bounded by four lines (can be named).
Gutter / gap	Space between tracks (`gap`, `row-gap`, `column-gap`) — only *between* tracks, never at the edges.
Explicit grid	The tracks you declared with `grid-template-*`.
Implicit grid	Extra tracks the browser creates when items are placed outside the explicit grid or outnumber its cells.

Grid behaviours to know up front

Like flex items, grid items are blockified, `float` is ignored, margins don't collapse, and `z-index` works without `position` (Chapter 7). Grid is **layout-in** (container tracks first, items fall into them); flexbox is **content-out** (items' sizes drive layout) — the key to 9.8.

9.2 Defining tracks

What it is

`grid-template-columns` / `grid-template-rows` list each track's size, space-separated:

Value	Meaning
`200px` , `10rem` , `30%`	Fixed size (percent of the container)
`fr`	A **fraction of the leftover space** after fixed tracks and gaps
`auto`	As a maximum: content size, but stretches to fill leftover space when there are no `fr` tracks (with `align/justify-content: normal`). As a minimum: the item's automatic minimum (usually min-content, so content isn't crushed)
`min-content`	As narrow as the content can get (longest word)
`max-content`	As wide as the content wants, no wrapping
`fit-content(150px)`	`max-content` , capped at 150px (and at least `min-content`) — never grows beyond content or cap
`minmax(min, max)`	Any size between min and max, e.g. `minmax(200px, 1fr)`

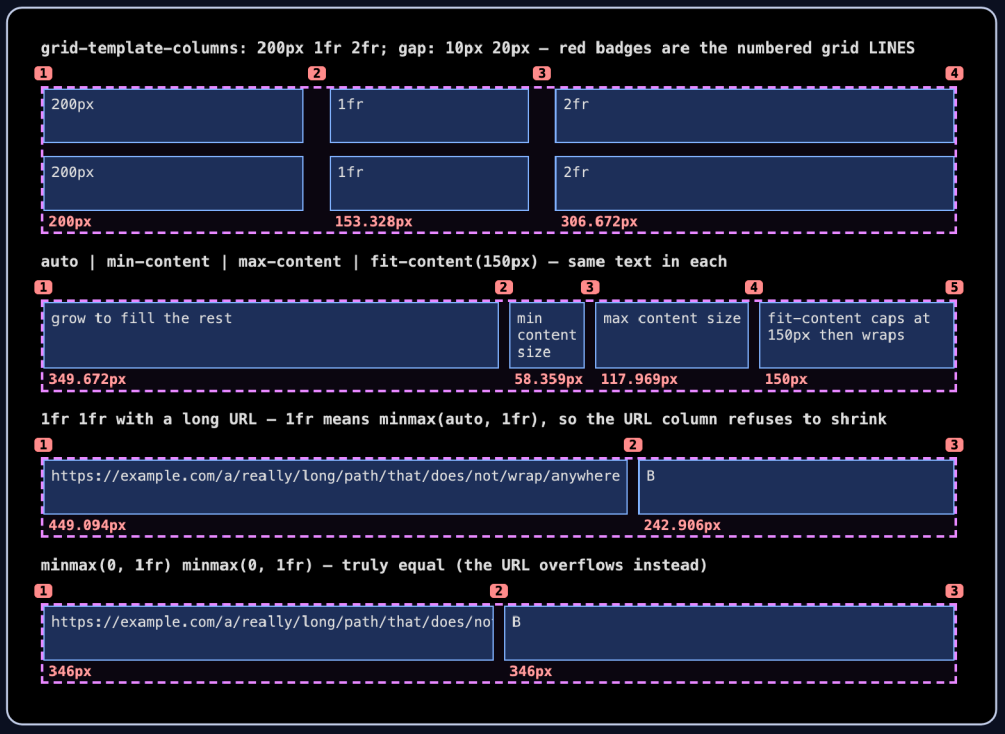


Figure 9.1 — `code/01_tracks.html` . Line numbers and sizes are drawn by a script from the resolved track list (absolutely positioned, so layout is unaffected).

How `fr` is calculated

1. Start with the container's inner size (700px).
2. Subtract every **fixed** track and gap: `200px 1fr 2fr` with `column-gap: 20px` → 700 − 200 − 2 × 20 = **460px**.
3. Divide by Σ `fr` (1 + 2 = 3): **1fr = 153.33px**, 2fr = 306.67px. (Container 1000px, `250px 1fr 1fr 1fr` , 10px gaps → (1000 − 250 − 30)/3 = 240px.)
4. A track whose share would fall below its minimum is fixed at that minimum and the rest re-split (like flexbox's freeze loop, §8.3).

`code/01_tracks.mjs` prints each declared value beside what Chrome resolved:

```
container content width: 700px
t1  200px 1fr 2fr                                   -> 200px 153.328px 306.672px
t2  auto min-content max-content fit-content(150px)  -> 349.672px 58.3594px 117.969px 150px
t3  1fr 1fr                                         -> 449.094px 242.906px
t4  minmax(0px, 1fr) minmax(0px, 1fr)               -> 346px 346px

by hand, t1: 1fr = 460 / 3 = 153.333px, 2fr = 306.667px (Chrome stores 1/64 px: 153.328125, 306.671875)
```

The tiny differences are Chrome's 1/64-pixel layout units, as in flexbox.

`fr` vs `%`

`%` tracks share the *whole* container and ignore gaps, so `50% 50%` plus a gap overflows (a horizontal scrollbar). `fr` shares only what's left after gaps.

The `1fr` minimum trap (t3 vs t4)

`1fr` is shorthand for **`minmax(auto, 1fr)`**, and the `auto` minimum means "never narrower than the item's min-content". In t3 a long unbreakable URL forces its column to 449px, leaving 242.9px — unequal. It's the grid twin of flexbox's `min-width: auto` (§8.4).

Fix: `minmax(0, 1fr)` (t4) lets the track shrink below its content — exactly 346px each — and handle the overflow with `overflow`, `overflow-wrap: anywhere` or `min-width: 0` on the item. For columns that **must** be equal (dashboards, code-and-preview splits): `repeat(3, minmax(0, 1fr))`.

Common mistakes

- `%` tracks with `gap` → horizontal scrollbar.
- Expecting `1fr 1fr` to be equal whatever the content.
- Forgetting rows default to `auto` — a row only exceeds its content height if sized, or if the container has a height and `fr` rows.

9.3 `repeat()`, `auto-fill` and `auto-fit`

What it is

`repeat(count, tracks)`: `repeat(4, 1fr)` = `1fr 1fr 1fr 1fr`; `repeat(2, 100px 1fr)` = `100px 1fr 100px 1fr`. The count can instead be **`auto-fill`** or **`auto-fit`** — "as many as fit". With `minmax()` this gives a responsive grid with **no media queries** (Chapter 10), the RAM pattern (**R**epeat, **A**uto, **M**inmax):

```
.gallery { display: grid; gap: 10px;
           grid-template-columns: repeat(auto-fill, minmax(120px, 1fr)); }
```

How it works

1. Tracks of the *minimum* size that fit: **n = floor((width + gap) / (min + gap))**.
2. n tracks are created; the `1fr` maximum stretches them to share the leftover.
3. Items auto-place; extras wrap onto new rows.
4. The difference shows only with **fewer items than tracks**:
    - `auto-fill` keeps the empty tracks — items keep their one-column width, space reserved on the right.
    - `auto-fit` **collapses** empty tracks (and their gaps) to 0, so items stretch to fill the row.

Auto-repeat needs a definite size (a fixed length or a `minmax` with a fixed minimum) to count tracks, so `repeat(auto-fill, 1fr)` is invalid.

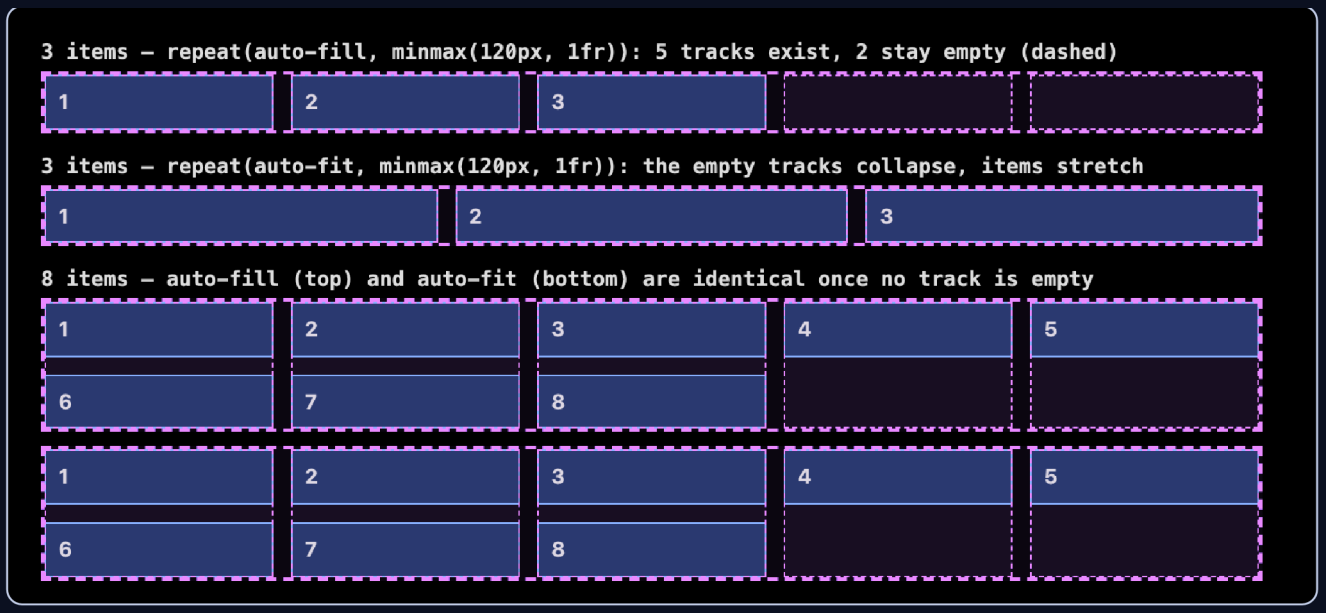


Figure 9.2 — code/02_auto_fill_fit.html. Every track, including empty ones, is outlined in dashed purple.

Output of `code/02_auto_fill_fit.mjs` (the four identical 400px lines merged):

```
container 700px, gap 10px, minmax(120px, 1fr):
  fill3  132px 132px 132px 132px 132px
  fit3   226.656px 226.672px 226.672px 0px 0px
  fill8  132px 132px 132px 132px 132px
  fit8   132px 132px 132px 132px 132px
container 400px, gap 10px, minmax(120px, 1fr):
  fill3, fit3, fill8, fit8  (all identical)  126.656px 126.672px 126.656px
by hand, 700px: n = floor(710 / 130) = 5, track = (700 - 40) / 5 = 132.000px
by hand, 400px: n = floor(410 / 130) = 3, track = (400 - 20) / 3 = 126.667px
by hand, auto-fit with 3 items at 700px: empty tracks and their gaps collapse -> (700 - 20) / 3 = 226.667px
```

In `fit3` the collapsed tracks appear as `0px`.

Which one to use

Want	Use
Cards keep a consistent size even with only one or two	`auto-fill`
A few items should stretch to fill the row	`auto-fit`

Guarding the minimum on narrow screens

If the container is narrower than the minimum (120px track in a 100px column), the single track overflows. Write `min(120px, 100%)`: `repeat(auto-fit, minmax(min(160px, 100%), 1fr))` (used in 9.9).

Self-check

1. Container 1000px, gap 20px, `repeat(auto-fill, minmax(200px, 1fr))`? → floor(1020/220) = 4 columns of (1000 − 60)/4 = **235px**. With 2 items and `auto-fit`? → (1000 − 20)/2 = **490px** each.

9.4 Placing items

By default each item takes the next free cell (9.5). To place one, name its **start and end lines** on each axis.

Line-based placement

Property	Example	Meaning
`grid-column-start` / `grid-column-end`	`1` / `3`	Start at column line 1, end at 3 (2 columns)
`grid-column` (shorthand)	`1 / 3`	Same
`span n`	`grid-column: span 2`	2 tracks wide, wherever auto-placement puts it
Negative lines	`grid-column: 1 / -1`	First to **last** line: full width
`grid-row`	`1 / span 2`	From row line 1, 2 rows tall
`grid-area` (4 values)	`3 / 2 / 4 / 5`	**row-start / column-start / row-end / column-end** (anticlockwise from the top, unlike `margin`)

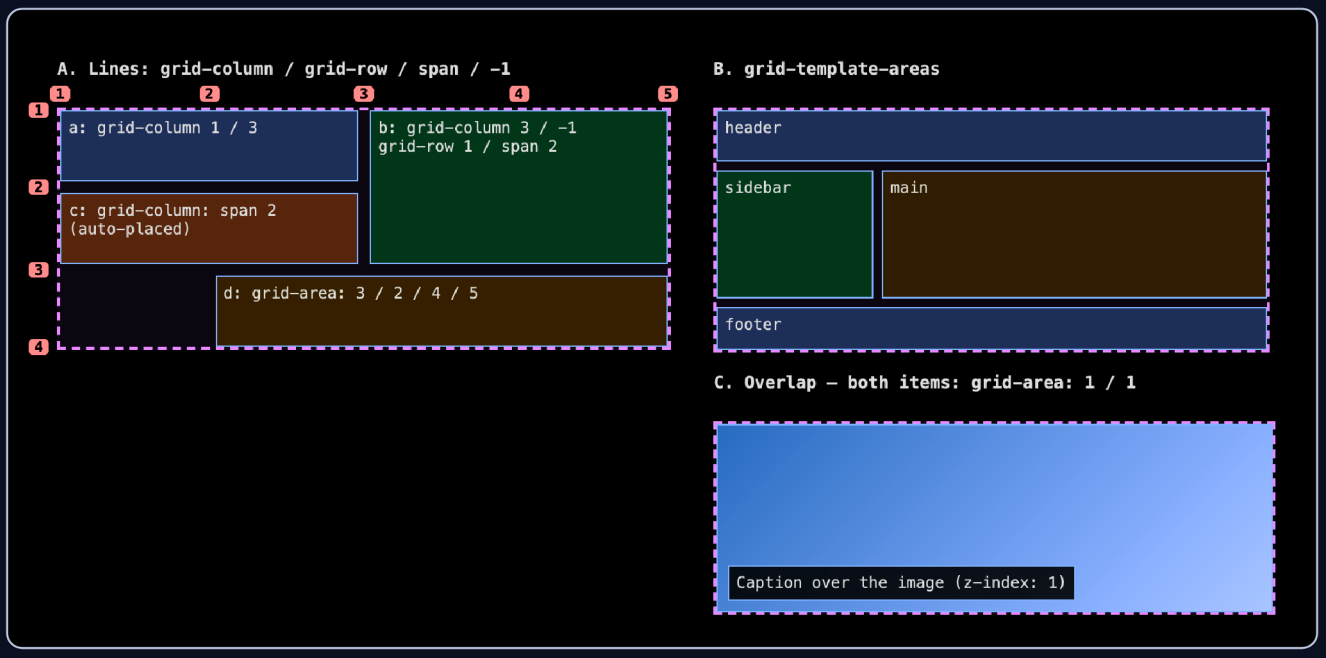


Figure 9.3 — code/03_placement.html.

```
#lines    { grid-template-columns: repeat(4, 1fr); grid-template-rows: repeat(3, 50px); gap: 8px; }
#lines .a { grid-column: 1 / 3; }
#lines .b { grid-column: 3 / -1; grid-row: 1 / span 2; }
#lines .c { grid-column: span 2; }           /* only a span: auto-placed into the first free spot */
#lines .d { grid-area: 3 / 2 / 4 / 5; }
```

code/03_placement.mjs maps each item back to line numbers:

```
a: columns 1 -> 3, rows 1 -> 2
b: columns 3 -> 5, rows 1 -> 3
c: columns 1 -> 3, rows 2 -> 3
d: columns 2 -> 5, rows 3 -> 4
```

`-1` resolved to line 5. Negative numbers count from the end of the **explicit** grid only, so `1 / -1` doesn't reach implicit columns.

Named lines

Name lines in square brackets and place by name — robust when numbers change:

```
.layout { grid-template-columns: [full-start] 1fr [content-start] minmax(0, 60rem) [content-end] 1fr [full-end]; }
.layout > *      { grid-column: content; }   /* "content" = content-start / content-end */
.layout > .bleed { grid-column: full; }      /* a full-width image or banner */
```

This **full-bleed** layout keeps text in a readable centre column while chosen elements run edge to edge.

`grid-template-areas`

The most readable page-layout tool: draw the layout as ASCII art, one string per row, and assign items by name.

```
#areas { grid-template-columns: 110px 1fr;
         grid-template-rows: 36px 90px 30px;
         grid-template-areas:
           "header  header"
           "sidebar main"
           "footer  footer"; }
.h { grid-area: header; } .s { grid-area: sidebar; } .m { grid-area: main; } .f { grid-area: footer; }
```

- Every string has the same number of cells.
- A name repeated in adjacent cells makes one area; each area must be a **rectangle** (`"a a" "a b"` is invalid — L-shaped).
- `.` (or `...`) marks an empty cell.
- Areas create named lines `header-start` / `header-end`, so `grid-column: header` works.
- Re-laying out at a breakpoint (a width where a media query switches layout) is just rewriting the strings (9.9).

Overlap

Items may share cells; later ones paint on top, and `z-index` changes that. Demo C puts an image and caption in `grid-area: 1 / 1` and aligns the caption bottom-left with `align-self: end; justify-self: start` — a hero banner with no `position: absolute`, so the container still grows with its content.

Accessibility note

As with flexbox `order`, placement doesn't change DOM order; keyboard and screen-reader users follow the source (§8.2). Keep source order logical.

9.5 The implicit grid and auto-placement

What it is

More items than explicit cells, or an item placed past the explicit grid, makes the browser add **implicit tracks** sized by `grid-auto-rows` / `grid-auto-columns` (default `auto` = content). E.g. `repeat(3, 1fr)` with 7 items and no rows → 3 implicit rows.

Auto-placement fills cells in order; `grid-auto-flow` sets how:

Value	Behaviour
`row` (default)	Fill each row left to right, then add rows
`column`	Fill each column top to bottom, then add columns
`row dense` / `column dense`	Also go **back** to fill earlier holes with later items that fit



Figure 9.4 — code/04_implicit.html. Orange items span 2 columns, green spans 3.

```
.flow   { grid-template-columns: repeat(4, 1fr); grid-template-rows: 50px; grid-auto-rows: 32px; }
.dense  { grid-auto-flow: row dense; }
.by-col { grid-template-rows: repeat(3, 32px); grid-auto-flow: column; grid-auto-columns: 70px; }
.outside      { grid-template-columns: repeat(2, 100px); grid-auto-columns: 50px; }
.outside .far { grid-column: 5; }
```

Output of code/04_implicit.mjs (`r2c3` = row 2, column 3):

```
row      columns: 99px 99px 99px 99px                        rows: 50px 32px 32px
         placement: 1@r1c1 2@r1c2 3@r2c1 4@r2c3 5@r2c4 6@r3c1 7@r3c4
dense    columns: 99px 99px 99px 99px                        rows: 50px 32px 32px
         placement: 1@r1c1 2@r1c2 3@r2c1 4@r1c4 5@r2c3 6@r3c1 7@r2c4
bycol    columns: 70px 70px 70px                             rows: 32px 32px 32px
         placement: 1@r1c1 2@r2c1 3@r3c1 4@r1c2 5@r2c2 6@r3c2 7@r1c3
outside  columns: 100px 100px 50px 50px 50px                 rows: 26px
         placement: 1@r1c1 2@r1c2 far@r1c5
```

- **row**: item 3 needs two columns; only column 4 is free in row 1, so it drops to row 2, leaving **r1c4 empty**. Row 1 is explicit (50px); rows 2–3 implicit (32px).
- **dense**: item 4 goes back to fill r1c4 — items now draw **out of source order**.
- **bycol**: three explicit rows, columns created as needed at 70px.
- **outside**: placing at column line 5 creates implicit columns 3–5 at 50px.

Useful settings

- `grid-auto-rows: minmax(100px, auto)` — at least 100px, growing with content; standard for card grids.
- `dense` suits galleries where order doesn't matter; avoid it where reading or tab order does.

9.6 Alignment

Grid uses the same Box Alignment properties as flexbox (§8.5), but **both axes have `justify-*` and `align-*`**, because items sit in 2-D cells:

- **inline axis** (row direction, left→right in English) → `justify-*`
- **block axis** (column direction, top→bottom) → `align-*`

Property	On	Aligns	Default
`justify-items` / `align-items`	container	every item **inside its cell**	`stretch` (unless the item has a set size or an aspect ratio, e.g. images → `start`)
`justify-self` / `align-self`	item	one item inside its cell (`place-self: center` centres just it)	inherits from `*-items`
`justify-content` / `align-content`	container	the **whole grid of tracks** in the container, when tracks are smaller than it	`normal` (tracks packed at start; `auto` tracks stretch)
`place-items`, `place-self`, `place-content`	—	shorthands: `align` value first, then `justify` (`place-items: end start` = align end, justify start)	—

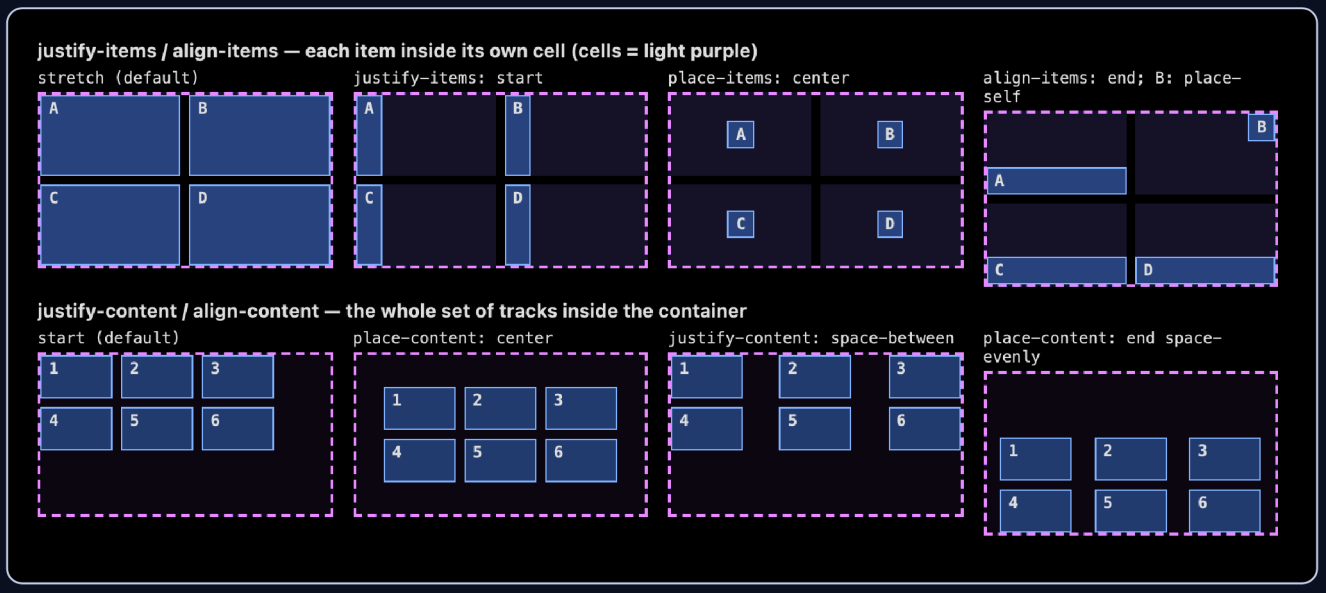


Figure 9.5 — code/05_alignment.html. Light-purple rectangles are the cells, showing each item's position inside its cell.

- `*-content` **only acts when there's free space** (fixed tracks smaller than the container); with `fr` tracks it does nothing.
- `space-between` / `-around` / `-evenly` in `*-content` **widen the effective gaps**; spanning items grow too.
- Auto margins work: `margin-left: auto` pushes an item to the end of its cell.

9.7 Subgrid

The problem it solves

A grid item that is itself a grid has its *own* tracks. Three pricing cards each lay out title, text and button, but card 2's longer title pushes its text lower — rows don't line up across cards. Before subgrid: fixed heights or JavaScript.

What it is

`grid-template-rows: subgrid` (or `-columns`) on a grid item spanning several parent tracks says: **don't create your own tracks on this axis — reuse the parent's tracks I sit on**. The card's children then join the parent's row sizing, so the tallest title in *any* card sets the title row for *all*. Defined in CSS Grid Level 2; **Baseline since September 2023** (Firefox 71, Safari 16, Chrome/Edge 117), widely available by 2026.

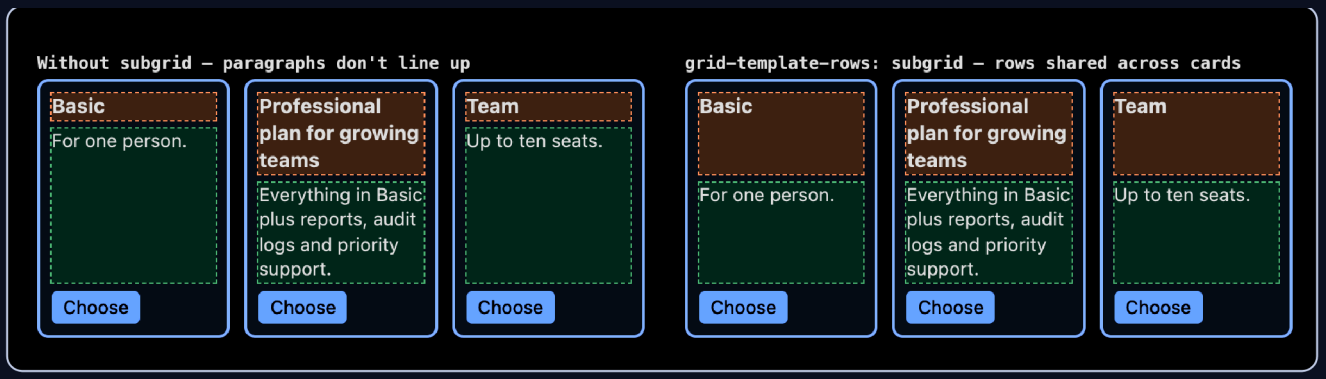


Figure 9.6 — `code/06_subgrid.html`. Orange dashed = title, green dashed = paragraph.

```
.sub       { display: grid; grid-template-columns: repeat(3, 1fr);
             grid-template-rows: auto auto auto; row-gap: 10px; }  /* the parent owns 3 rows */
.sub .card { display: grid; grid-row: span 3;                       /* each card covers all 3 */
             grid-template-rows: subgrid; row-gap: 6px; }           /* ...and reuses them */
```

Measured by `code/06_subgrid.mjs` (top edges of each paragraph and link):

```
plain  <p> tops: 76.9, 114.7, 76.9    link tops: 190.9, 190.9, 190.9
sub    <p> tops: 114.7, 114.7, 114.7  link tops: 190.9, 190.9, 190.9
```

Without subgrid the links happen to align (each card's middle row is `1fr` and cards stretch to equal height) but paragraphs don't; with subgrid every row is shared.

Details

- The subgrid item must **span** the parent tracks it shares (`grid-row: span 3`).
- It inherits the parent's gaps but may override them (`row-gap: 6px`).
- You can subgrid one axis and define your own tracks on the other.
- Parent line names are available inside.
- Uses: card rows, forms whose labels and inputs sit in separate `<fieldset>`s (Chapter 5), lists whose items have aligned columns (a `<ul>` of rows with `grid-template-columns: subgrid`).

9.8 Grid vs Flexbox

Question	Flexbox	Grid
Dimensions	One — a row **or** a column	Two — rows **and** columns
Who decides sizes?	**Content-out**: items' content and flex factors	**Layout-in**: the container's track definitions
Items in different rows line up in columns?	No — each flex line is independent	Yes — that's the point
Wrapping	Each line sizes itself; a lonely last item stretches	Items drop into the same columns; last row keeps the column width
Overlapping items	Needs negative margins or positioning	Place two items in the same cell
Gaps	`gap`	`gap`
Reordering	`order`, `*-reverse`	Line/area placement, `order`, `dense`
Typical jobs	Navbars, toolbars, button groups, tags, media objects, centring one thing, "push the last item right"	Page layouts, galleries, card grids, dashboards, form layouts, anything designed on a grid
Aligning content across siblings	Not possible	`subgrid`

Content-driven row of different-sized things → flexbox; things must line up both ways, or track sizes known up front → grid; usually both, nested (grid for the page and gallery, flexbox in each card's footer).

> **30-second answer:** "Flexbox is one-dimensional and content-out: items share space by content and flex factors. Grid is two-dimensional and layout-in: I define rows and columns and place items, so things align across rows. Grid for page structure and card grids, flexbox for components like navbars and button groups; often nested."

9.9 Patterns

Holy grail layout (header, nav, main, aside, footer)

```
/* mobile first: one column, in source order */
.page { display: grid; gap: 8px; min-height: 100dvh;          /* demo uses 260px */
        grid-template-areas: "header" "nav" "main" "aside" "footer";
        grid-template-rows: auto auto 1fr auto auto; }
.page > header { grid-area: header; }  .page > nav    { grid-area: nav; }
.page > main   { grid-area: main; }    .page > aside  { grid-area: aside; }
.page > footer { grid-area: footer; }

/* three columns when there's room */
@media (min-width: 640px) {
  .page { grid-template-columns: 150px 1fr 170px;
          grid-template-rows: auto 1fr auto;
          grid-template-areas:
            "header header header"
            "nav    main   aside"
            "footer footer footer"; }
}
```

The `1fr` row gives `main` all spare height, so the footer sits at the screen bottom (the §8.7 sticky footer, done with grid). Media queries: Chapter 10.

Responsive card grid (RAM)

```
.cards { display: grid; gap: 10px;
         grid-template-columns: repeat(auto-fit, minmax(min(160px, 100%), 1fr)); }
```

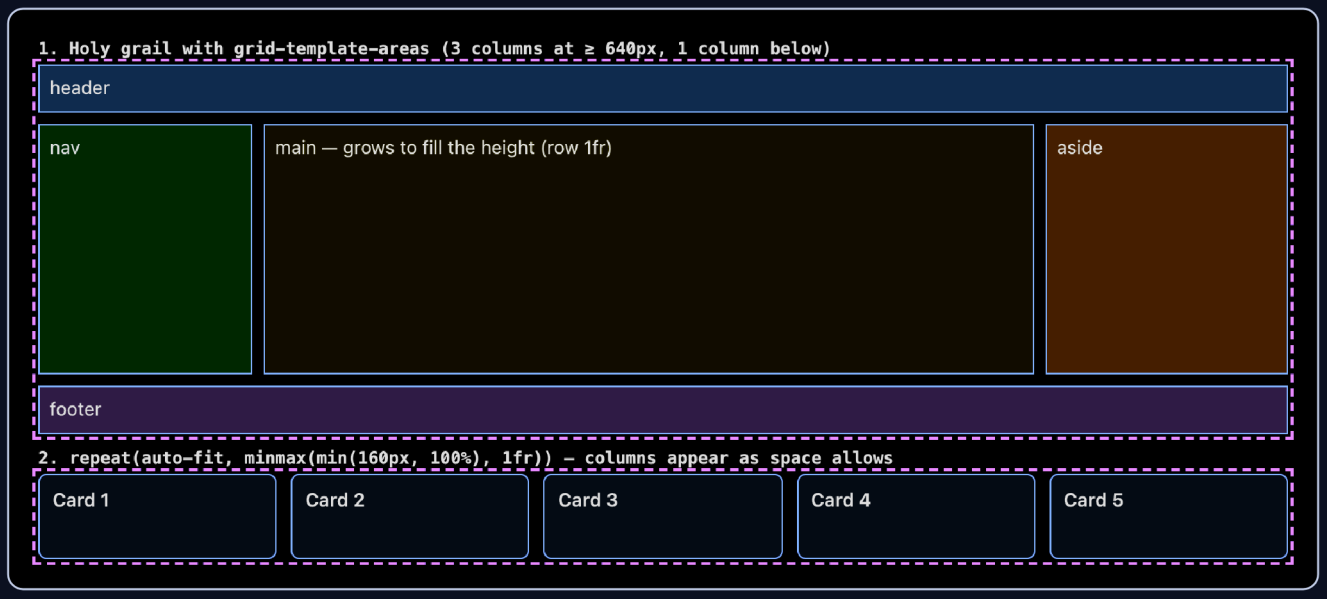




Figure 9.7 — `code/07_patterns.html` at 900px and 400px wide.

`code/07_patterns.mjs` :

```
viewport 900px
  holy grail: 150px 540px 170px
  cards     : 167.188px 167.203px 167.203px 167.203px 167.188px
viewport 400px
  holy grail: 376px
  cards     : 183px 183px
```

At 900px the container is 876px: 876 − 150 − 170 − 2 × 8 = 540 for `main` ; floor(886/170) = 5 card columns. At 400px the media query no longer matches (one 376px column) and 2 card columns fit; card 5 keeps the column width on its own row — what flex-wrap couldn't do (§8.6).

Other patterns worth knowing

Pattern	Grid recipe
Centre anything	`display: grid; place-items: center`
Overlay / hero caption	Image and caption both `grid-area: 1 / 1` ; align the caption with `align-self` / `justify-self`
Full-bleed article	Named lines `[full-start] 1fr [content-start] minmax(0, 60rem) [content-end] 1fr [full-end]` (9.4)
Form with aligned labels	`grid-template-columns: max-content 1fr` , one label + input per row
Sidebar that collapses	`grid-template-columns: fit-content(250px) minmax(0, 1fr)`
12-column design-system grid	`repeat(12, minmax(0, 1fr))` + `grid-column: span 4` on items

Masonry: not yet

Pinterest-style "masonry" (columns of uneven-height items) is being standardised in CSS Grid Level 3 as `display: grid-lanes`. As of 2026 it shipped in Safari (26.4) and is behind flags in Chrome and Firefox — not Baseline. Use CSS multi-column (`columns: 3`) or a JS library, and wrap experiments in `@supports (display: grid-lanes)` (a feature query).

9.10 Debugging, pitfalls and interview prep

Debugging

All major browsers show a **grid badge** beside grid containers in DevTools' Elements/Inspector panel (F12); toggling it overlays positive and negative line numbers, track sizes, gaps and area names. Firefox's grid inspector pioneered this and is still the most detailed.

Pitfalls at a glance

Symptom	Cause	Fix
`1fr` columns are unequal / grid overflows on mobile	`1fr` = `minmax(auto, 1fr)`; long words, `<pre>`, images or tables set the minimum	`minmax(0, 1fr)`; `min-width: 0` or `overflow` on the item (e.g. wrap a wide table in `overflow-x: auto`)
Horizontal scrollbar with `%` columns	`%` tracks ignore gaps	Use `fr`
Single auto-fill column overflows narrow screens	Minimum larger than the container	`minmax(min(200px, 100%), 1fr)`
Item lands in an unexpected extra row/column	Placed beyond the explicit grid → implicit tracks; or a misspelt area name (unknown names create implicit lines)	Check line numbers in DevTools; spell area names consistently
`grid-template-areas` ignored	Unequal cell counts or a non-rectangular area — the whole declaration is invalid	Fix the ASCII art
An image doesn't fill its cell	Items with an aspect ratio default to `start`, not `stretch` — in Chrome a 100×50 image in a 200×300 cell stays 100×50	`width: 100%; height: 100%; object-fit: cover` (Chapter 11)
`justify-content` does nothing	`fr` tracks leave no free space	Fixed tracks, or align items instead
Tab order jumps around	`dense`, placement or `order` moved items visually	Keep DOM order = visual order

In the interview

30-second answer: "Grid is two-dimensional layout. On the container I define tracks with `grid-template-columns` / `-rows` using lengths, `fr` for fractions of leftover space, `minmax` and `repeat` — `repeat(auto-fill, minmax(200px, 1fr))` gives a responsive grid with no media queries. Items are auto-placed, or placed by line numbers, spans or named `grid-template-areas`. Extra items create implicit tracks sized by `grid-auto-rows`. Alignment uses `justify-*` for the inline axis and `align-*` for the block axis, and `subgrid` lets nested grids share the parent's tracks so cards line up."

- *How is `fr` calculated?* → Container minus fixed tracks and gaps, divided by total `fr`; tracks below their minimum are fixed and the rest re-split. 700 − 200 − 40 = 460 → 1fr = 153.33.
- *A layout that reorders at a breakpoint?* → Named areas rewritten in a media query — keeping DOM order sensible for keyboard and screen-reader users.
- *What is subgrid for?* → Aligning the insides of sibling components (card titles, form labels) to shared tracks.

Chapter 9 TL;DR

- **Grid** = two-dimensional, layout-in. `display: grid` → direct children are items. Lines (1…n+1, −1 backwards), tracks, cells, areas, gaps; explicit vs implicit grid.
- **Tracks**: px/%/ `fr` / `auto` / `min-content` / `max-content` / `fit-content()` / `minmax()`. **`fr` = share of space left after fixed tracks and gaps** (700 − 200 − 40 = 460 → 153.33 / 306.67, verified). `1fr` = `minmax(auto, 1fr)` → `minmax(0, 1fr)` for truly equal columns.
- **RAM**: `repeat(auto-fill|auto-fit, minmax(min(X, 100%), 1fr))`; n = floor((W + gap) / (X + gap)). auto-fill keeps empty tracks, auto-fit collapses them.
- **Placement**: `grid-column: 1 / 3`, `span 2`, `1 / -1` (full width), `grid-area: row-start / col-start / row-end / col-end`, named lines, `grid-template-areas` ASCII art (rectangles only, `.` = empty). Items can overlap in one cell.

- **Implicit grid**: `grid-auto-rows/columns` size extra tracks; `grid-auto-flow: row | column | dense` (dense breaks visual/DOM order).
- **Alignment**: `justify-*` = inline axis, `align-*` = block axis; `*-items` / `*-self` inside cells, `*-content` for the whole grid when tracks leave free space; `place-*` shorthands; `place-items: center` centres.
- **Subgrid** (Baseline since Sept 2023): `grid-template-rows: subgrid` on a spanning item reuses the parent's tracks so card internals align.
- **Grid vs Flexbox**: grid for page structure and uniform grids; flexbox for content-sized rows inside components; nest them.
- **Masonry** (`display: grid-lanes`) is Safari-only in 2026 — not Baseline.

Next: Chapter 10 — Responsive Design.